AI Technical Book Reader

https://gemini.google.com/app/91b6e29f000b3de1

User prompt: i have a few programming books as pdf files, but I know from experience a lot of content is just talking shit instead of actually learning something. But in this era we have AI! I was thinking to build a system that uses cursor sdk for example, receives a pdf file, splits it into chapters or even smaller parts and from each section builds a nice interactive board with things i should really remember, things that may be usefull at some point, ofcourse all the schemas it foinds with nice animations and all the code examples in the book explained, exercises and more. do you have a better idea? also each board must also specify the number of pages it covers so i can check. do you think a dashboard may be too token expensive? maybe it doesnt even need to be html? the app i am thinking to use tauri and with a nice clean, very minimalist user interface with a place where i can drag the file i want, one at a time only, a button that says speed up learning! a textbox where i can put the output path for the artifacts and a drop down from where i can select what model to use. ofcourse will have settings too to change theme and where i will add the ai sdk key. the app will be strictly for more technical books, so no fantasy or other stuff like that because those are meant to read and enjoy while technical books are ment to teach you stuff without wasting time

Response: Having the AI generate a full HTML dashboard for every chapter is a massive token trap—**output tokens are typically 3 to 5 times more expensive than input tokens**, and forcing a model to write HTML tags, CSS styling, and animation scripts over and over means paying for thousands of tokens of repetitive layout code instead of actual book insights.

You can get the exact same (or better) interactive result for **roughly 80% fewer output tokens** by separating the **visual interface** from the **extracted knowledge**.

1. The Better Format: One Built-In Viewer, Tiny Data Files

Instead of asking the AI to build a web page for every section, build the interactive board **once** inside your Tauri app (or drop a single lightweight viewer file into your output folder) and have the AI output only **structured JSON or Markdown**.

Output Approach	Avg. Output Tokens per Chapter	Animations & Diagrams	Reliability
Raw HTML / CSS / JS	4,000 – 8,000+ tokens	High risk of broken layouts or glitchy scripts	Low (models hallucinate styling)
Plain Markdown (`.md`)	600 – 1,200 tokens	Static only (standard Mermaid charts)	Very High
Structured JSON + Markdown (Best)	**800 – 1,500 tokens**	**Step-by-step animated flows driven by your UI**	**100% consistent look & feel**

How the JSON + Markdown hybrid works:

- Your Tauri app has a clean built-in **Board Viewer** (and can also export a single self-contained `index.html` viewer to your chosen output path that reads the generated `.json` chapter files).
- For each chapter, the AI only outputs the raw text, code explanations, and a tiny node-and-arrow list for diagrams.
- **Animated Schemas for Cheap:** Instead of asking the AI to write animation code, ask it to output a simple list of steps for each diagram (e.g., `Step 1: Client -> Auth Server`, `Step 2: Auth Server -> Database`). Your Tauri interface renders those boxes automatically and gives you a **"Next Step" / "Play"** button that highlights each box and animates a glowing line between them. The AI writes 15 lines of text; you get a smooth interactive animation every time.

2. Five Upgrades to Make the System Smarter

1. **Inject `[PAGE X]` Markers Locally Before Calling the AI** If you just extract text from a PDF and send a chunk to the AI, the model has no idea what page it is looking at—it will guess or hallucinate page numbers. Have Tauri's local PDF reader stamp `--- [PAGE 42] ---` at the exact boundary of every page before sending the text out. The AI can then reliably label the board **"Covers Pages 42–56"** and even tag individual code blocks or schemas with **"Found on Page 49"** so you can verify the original book in two seconds.
2. **Add a 1-Second "Chapter Filter" Before Burning Tokens** Even the best technical books have filler chapters (*"Chapter 1: The History of Databases"*, *"Chapter 2: Installing Your Editor"*). When you drop a PDF in and click **"Speed up learning!"**, have Tauri instantly read the PDF's Table of Contents locally (zero AI cost) and pop up a quick checklist showing each chapter and its page count. Uncheck the fluff chapters you already know, and only spend tokens on the meat of the book.
3. **Use Vision Only on Pages with Real Diagrams** Standard PDF text extraction completely ignores image-based diagrams and architecture charts. Have your local Tauri extractor check if a page contains embedded images or vector drawings. If a page is pure text and code, send cheap text tokens. If a page has a diagram, automatically take a snapshot of that specific page and send it to the model's vision input so it can read the visual diagram and turn it into an animated step-by-step flow.
4. **The 3-Tier Board Layout** Structure every generated board into four fixed zones so your eyes always know where to look:
    - **Header Bar:** Chapter title, **Page Range (e.g., Pages 84–102 · 19 pages compacted)**, and estimated reading time saved.
    - **Core Vault (Must Remember):** The 20% of concepts that give you 80% of the value—hard rules, mental models, and gotchas.
    - **Back-Pocket Reference (Useful Later):** Edge cases, configuration flags, or niche patterns you don't need to memorize today, just know they exist.

    - **Interactive Lab (Schemas, Code & Exercises):** Step-through animated diagrams, side-by-side code breakdowns (original book code on the left, plain-English annotations on the right), and hands-on challenges.

5. **"Spot the Bug" Exercises Instead of Trivia Quizzes** Multiple-choice questions at the end of a chapter are easy to ignore. Have the AI take an actual code example from that chapter, secretly introduce a realistic mistake based on a rule the chapter just taught, and ask you: *"Why would this fail in production?"* or *"Which line violates the pattern from Page 64?"*

3. Tech Stack & The "Cursor SDK" Question

Cursor doesn't offer an embeddable SDK for standalone desktop apps (it only has a CLI agent meant for local workspaces, which won't smoothly wire into your custom API key box and model dropdown).

For a **Tauri** app with a model dropdown and custom API keys, the cleanest stack is:

- **AI Layer: Vercel AI SDK** (`ai` package in your frontend/TypeScript layer) or direct REST calls from Rust. The Vercel AI SDK lets you swap between OpenAI, Anthropic, and Google Gemini models from a single dropdown with one line of code, and it enforces **Structured Outputs** so the AI is guaranteed to return the exact board format your UI expects.
- **PDF Parsing (Local in Rust or JS):** `pdfjs-dist` (in the frontend) or `pdf-extract` / `lopdf` (in Rust) to split chapters by bookmarks and inject `[PAGE X]` markers without freezing the UI.
- **Animated Diagrams: React Flow** (or **Svelte Flow**) + **Dagre** (for automatic box layout). The AI just names the boxes and connections, and the library draws and animates them cleanly in dark or light mode.
- **Output Folder Artifacts:** Save each processed chapter as `03-memory-management.board.json` alongside a clean `03-memory-management.md` summary in your chosen output path. That way, you can open them inside your Tauri app for the full animated board experience, or read the Markdown files directly anywhere else.

User prompt: sounds good what you proposing, but the summary i do not want it to be something like, i have this 400+ pages programming book and it generates me only 10 pages. i still expect it to be pretty throughly and indepth because i know ai is very good at summaries and extracting data. The viwer idea sounds pretty good, curios how would look like. so basically after i press the button in my UI can open a separate section where i can see the progress and where the viwer will be loaded. for schemas and other stuff like this maybe would be good if it extracts them directly from the book? would this save more tokens or waste more? for text to let ai extract it or have an mit specialized library to do that and feed into ai the text directly?

Response: To keep a 400-page technical book from shrinking into a superficial 10-page summary, you have to change **how you slice the book** before the AI ever sees it.

1. Why AI Turns 400 Pages into 10 (And How to Stop It)

If you feed an entire 35-page chapter into an AI at once, the model hits its output ceiling and gets lazy—it compresses 35 pages of deep lessons into a 1-page bulleted list.

To get **100% of the technical depth with 0% of the author's rambling**, do two things:

- **Slice by Sub-Sections (3 to 5 Pages at a Time), Not Full Chapters:** A 400-page book usually has 12–15 chapters, which is too broad, but it has **80 to 110 sub-sections** (like `2.1`, `2.2`, `2.3`). When the AI only has to process 4 pages at a time, it has plenty of room to extract every single concept, rule, edge case, and code block without skipping anything. Across 100 sub-sections, your 400-page book becomes roughly **120–150 boards of pure, dense meat**—only the author's personal stories, repetitive intros, and filler get deleted.
- **Ban the Word "Summarize" in Your Instructions:** When an AI hears *"summarize"*, it throws away details. Instead, instruct it to perform **"Complete Technical Extraction"**:

    > *"Delete conversational storytelling, historical fluff, and repetitive introductions. Keep 100% of the technical rules, every single code snippet, all numbers/limits, and every edge case or warning."*

2. Text Extraction: Local MIT Library vs. AI

Always use a local MIT-licensed library to extract text first. Never send raw PDF pages to the AI just to read text.

- **Why AI text extraction is a trap:** Sending a raw 400-page PDF directly into an AI model treats pages like images or heavy documents, burning **400,000 to 600,000+ input tokens** just to read the file, and takes minutes.
- **Why a local MIT library wins:** It extracts all 400 pages locally on your PC in **under 1 second for $0**, lets you stamp `[PAGE 42]` markers automatically, and lets you detect code blocks before calling the AI.

Best MIT-Licensed Libraries for Your Tauri App:

- **`pdf_oxide` (Rust & JS/TS — MIT License):** Built natively in Rust, so it plugs straight into Tauri's backend. It extracts text, pulls out embedded images, and converts pages directly into structured Markdown at roughly 0.8 milliseconds per page.
- **`pdfjs-dist` / `pdf.js-extract` (TypeScript — Apache-2.0 / MIT wrapper):** Runs Mozilla's PDF engine and gives you the exact `x, y` position and font name of every line on the page. That font data is a superpower for technical books: whenever the font switches to a monospace font (like `Courier` or `Consolas`), your app immediately knows **that block is a code example** without spending a single token.

3. Book Schemas: Extracting Directly vs. AI Generation

Extracting the diagrams directly from the PDF saves massive amounts of tokens and protects you from the AI misreading a complex chart.

Here is how the token math compares across three approaches:

Approach	Token Cost per Diagram	Accuracy	Interactive Animation?
1. Send page image to AI Vision + generate animated flow	~1,200 input + ~400 output tokens	Medium (can misread tiny arrows)	Always on
2. Extract original image locally (`pdf_oxide`)	**0 tokens**	**100% (exact book figure)**	Static image only
3. Hybrid: Local Crop + On-Demand "Animate" Button	**0 tokens upfront** (~300 tokens only when clicked)	**100% + Interactive**	**On-demand**

Why Approach #3 (The Hybrid) is the sweet spot:

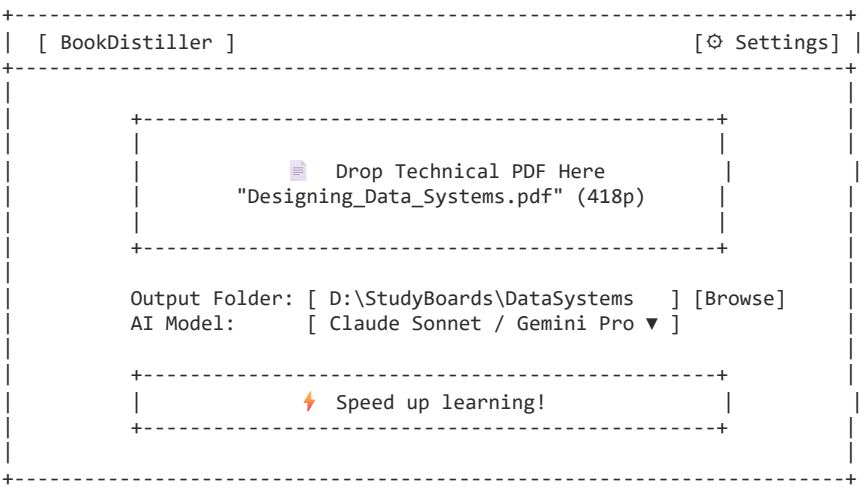
1. Your local PDF library automatically extracts every figure/diagram on pages 42–46 and saves them to your output folder (`page-44-fig-1.png`) for **zero tokens**.
2. Your Viewer displays the crisp original book diagram right inside the board so you see the exact schema the author drew.
3. In the corner of every extracted diagram, put a small **"✨ Turn into Animated Flow"** button.
    - Also, remember that technical books almost always explain a diagram in the text right below it (*"As shown in Figure 2.4, request A goes to B..."*). So even if you want animations generated automatically, the AI can usually build the animated step-by-step boxes **just from reading the surrounding text**, without ever paying vision token prices for the image!

4. How the Minimalist UI & Viewer Transition Works

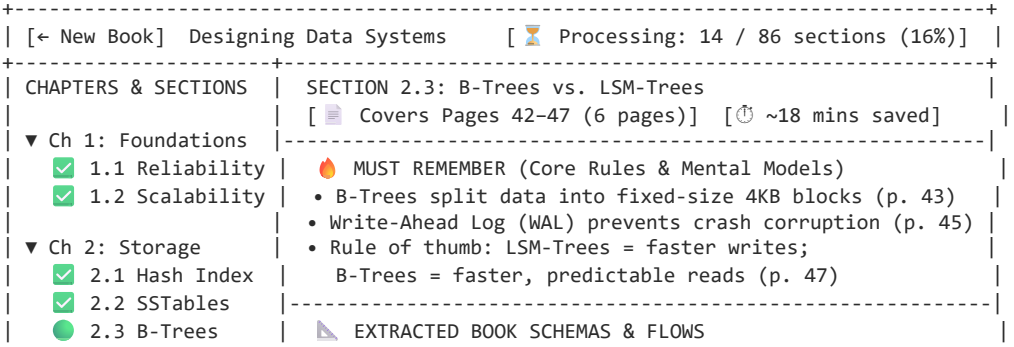
You don't need to wait 10 minutes for all 400 pages to finish before you start reading. As soon as you click **"Speed up learning!"**, your window slides from the minimalist setup view into the **Live Study Workspace**, streaming sections in one by one so you can read Section 1.1 four seconds after clicking the button.

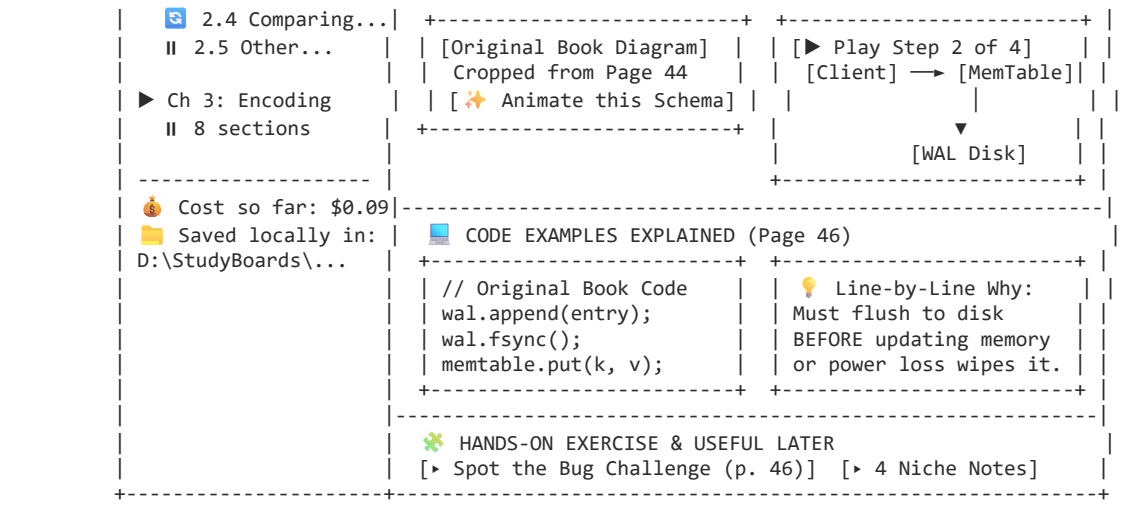
Here is how the two views flow together:

View 1: The Minimalist Launchpad (Before Clicking)



View 2: The Live Progress & Board Viewer (Opens After Clicking)





Why this workflow feels effortless:

- **Pause & Resume Anytime:** Because each sub-section is saved immediately to your output folder as a lightweight `.board.json` file, if you close the app at 16% progress, you never lose what you already paid for. Next time you open that folder, it instantly loads the completed boards and lets you resume from Section 2.4.
- **Click-to-Verify Original Page:** Clicking the `[ 📄 Covers Pages 42–47]` badge (or any `(p. 44)` tag) can open a quick side drawer showing the actual PDF page from your book so you can verify the original page in one click if anything ever looks suspicious.

Sources

1. GitHub - yfedoseev/pdf_oxide: The fastest PDF library for Python
2. 7 PDF Parsing Libraries for Extracting Data in Node.js - Strapi

---

User prompt: I want the final stack you are proposing to use, tech and why and for what, structure of the app and everythign needed so I can start the dev. so basically like a prd

Response: Product Requirements Document (PRD): Technical Book Distiller

Core Goal: Transform 400+ page technical PDFs into a deep, interactive collection of **100 to 150 section-by-section study boards**—removing author rambling and filler while keeping 100% of the rules, architecture diagrams, code examples, edge cases, and page references.

1. Final Tech Stack (What, Why, and For What)

Every tool in this stack is open-source (MIT / Apache-2.0) and chosen to keep local work free while spending AI tokens only on pure knowledge extraction.

Layer	Technology (2026 Stable)	Why This Choice	What It Does in Your App
Desktop Shell	**Tauri 2.12** (Rust + Webview)	Uses the operating system's native window renderer instead of bundling a whole browser, keeping the app tiny and fast.	Handles drag-and-drop PDF files, local folder reading/writing, background PDF chopping, and encrypted API key storage.
Screen & Styling	**React 19 + TypeScript + Tailwind CSS + shadcn/ui**	Clean, minimalist look out of the box with instant dark/light theme switching and zero visual clutter.	Builds the Launchpad screen, the Chapter Filter checklist, the Settings window, and the 3-Tier Board Viewer.
Local PDF Text & Layout Reader	`pdfjs-dist` **(v6.4)**	Mozilla's battle-tested PDF reader. Gives exact page numbers, bookmark trees, text positions, and font names for $0.	Splits the book into 3–5 page sub-sections, stamps `[PAGE X]` markers, detects code blocks by monospace font, and renders the "Verify Original Page" side drawer.
Local Diagram & Image Extractor	`pdf_oxide` **(Rust) + Canvas Cropper**	Pulls embedded figures and vector drawings straight out of PDF pages in milliseconds without using AI vision tokens.	Saves every diagram from the book as a crisp `.png` inside your output folder (`assets/page-44-fig-1.png`) and links it to that page's board.

Layer	Technology (2026 Stable)	Why This Choice	What It Does in Your App
AI Connection Layer	**Vercel AI SDK 7 (`ai` v7)**	Lets your model dropdown switch between Claude, Gemini, OpenAI, OpenRouter, or even local coding harnesses like Cursor with a single setting, while forcing the AI to reply in your exact board layout.	Sends 3–5 page text chunks to your chosen model, streams progress live, tracks exact token cost, and guarantees the output matches your board template.
Animated Schemas	**React Flow 12.12 (`@xyflow/react`) + Dagre**	Turns a tiny list of boxes and arrows from the AI into a smooth, interactive diagram with automatic spacing.	Powers the **"Play Step-by-Step"** animated flows and the on-demand **"✨ Animate This Schema"** button.
Code Breakdown Display	**Shiki**	Uses the exact same color-highlighting engine as VS Code and supports line-by-line highlights and diff markers.	Renders the book's code examples side-by-side with plain-English explanations and powers the "Spot the Bug" exercises.
Local State & Settings	**Zustand** + **`@tauri-apps/plugin-store`**	Simple state tracking with zero boilerplate; Tauri's store saves settings safely on your machine.	Remembers your theme, preferred model, API keys, default output folder, and where you left off in a book.

2. Application & Output Folder Structure

A. Project Source Tree (`book-distiller/`)

```
book-distiller/
├── src-tauri/                        # Local background engine (Rust)
│   ├── src/
│   │   ├── main.rs                   # App entry point
│   │   ├── pdf_slicer.rs             # Reads bookmarks, stamps [PAGE X], crops figures
│   │   ├── storage.rs                # Saves .board.json, .md, and images to Output Path
│   │   └── settings.rs               # Secure local storage for API keys
│   └── Cargo.toml
│
├── src/                              # Minimalist User Interface (TypeScript + React)
│   ├── components/
│   │   ├── launchpad/
│   │   │   ├── DropZone.tsx          # Single-file PDF drag & drop area
│   │   │   ├── RunControls.tsx       # Output path input, Model dropdown, "Speed up learning!" button
│   │   │   └── ChapterFilterModal.tsx# Quick checklist to skip fluff chapters before spending tokens
│   │   ├── workspace/
│   │   │   ├── SidebarProgress.tsx   # Live section tree, progress bar, running cost counter
│   │   │   ├── BoardViewer.tsx       # Main 3-tier board display (Core Vault, Schemas, Code, Exercises)
│   │   │   ├── AnimatedSchema.tsx    # React Flow step-by-step diagram player
│   │   │   ├── CodeExplainer.tsx     # Side-by-side book code + line-by-line annotations
│   │   │   ├── SpotTheBugCard.tsx    # Interactive broken-code exercise with reveal button
│   │   │   └── PdfPageDrawer.tsx     # Pop-out drawer showing the exact original PDF page
│   │   └── settings/
│   │       └── SettingsModal.tsx     # Theme toggle (Dark/Light) + API Key inputs per provider
│   ├── ai/
│   │   ├── extractor.ts              # AI SDK 7 call logic + token cost calculator
│   │   ├── prompts.ts                # System instructions banning shallow summaries
│   │   └── boardSchema.ts            # Exact template the AI must fill out for every section
│   └── store/
│       └── useAppStore.ts            # Tracks active book, current section, and processing queue
└── package.json
```

B. What Gets Saved in Your Chosen Output Path

When you point the app to `D:\StudyBoards\Designing_Data_Systems`, it writes clean, portable files so you never lose progress if you close the app mid-book:

```
D:\StudyBoards\Designing_Data_Systems\
├── book-manifest.json                # Book title, total pages, list of sections, and completion status
├── assets/
│   ├── page-044-fig-1.png            # Original diagrams cropped directly from the PDF ($0 cost)
│   └── page-046-fig-2.png
├── boards/                           # Loaded by your built-in Tauri Viewer
│   ├── 01-1-reliability.board.json
│   ├── 01-2-scalability.board.json
│   └── 02-3-b-trees-vs-lsm.board.json
└── markdown/                         # Clean readable notes you can open in any text app
    ├── 01-1-reliability.md
```

```
├── 01-2-scalability.md
└── 02-3-b-trees-vs-lsm.md
```

3. End-to-End Workflow (How the App Runs)

Step 1: Minimalist Launchpad (Single File Input)

1. **Drag & Drop Zone:** Accepts **one `.pdf` file at a time**. Dropping a file immediately displays the book title, total page count, and detected chapter count.
2. **Output Path Box:** Textbox with a **Browse** button (auto-fills a subfolder named after the book inside your default study directory).
3. **Model Selector Dropdown:** Lists models grouped by speed/depth (e.g., *Claude 4.5 Sonnet*, *Gemini 2.5 Pro*, *Gemini 2.5 Flash*, *GPT-4.1*).
4. **Action Button:** Clicking **" ⚡ Speed up learning!"** opens a 2-second **Section Filter popup** listing all detected chapters and sub-sections with their page ranges. Preface, Acknowledgments, Index, and Appendix are auto-unchecked to save tokens.

Step 2: Local Slicing & Zero-Token Preprocessing

Before calling the AI, the local engine slices the checked chapters into **Sub-Sections of 3 to 6 pages each** (using PDF bookmarks first; if a chapter is longer than 6 pages without sub-bookmarks, it splits by heading font size).

- **Page Stamping:** Every page's text is wrapped with `--- [START PAGE 42] ---` and `--- [END PAGE 42] ---`.
- **Local Diagram Extraction:** Any embedded image or vector diagram box on pages 42–47 is saved to `assets/page-044-fig-1.png`, and a marker `[EXTRACTED_FIGURE: assets/page-044-fig-1.png on PAGE 44]` is placed into the text at that exact spot.

Step 3: Live Workspace Transition & Streaming Loop

The window slides into the **Live Study Workspace**:

- **Left Sidebar:** Shows all sub-sections (✅ `Done`, 🟢 `Reading Now`, 🔄 `Extracting...`, ⏸ `Queued`), a progress bar (`14 / 92 sections`), and a live cost counter (`$0.08 spent`).
- **Main Viewer:** As soon as Section 1.1 finishes (roughly 4–6 seconds after clicking start), it loads on screen so you can begin studying while the background worker processes Section 1.2, 1.3, and so on.
- **Instant Resume:** Before sending any section to the AI, the app checks if `boards/02-3-b-trees.board.json` already exists in your output folder. If it does, it loads it from disk for $0 and skips to the next unfinished section.

4. The Board Data Template & AI Instructions

A. The `.board.json` Template

Instead of asking the AI for bloated HTML, force the AI to fill out this compact structure for every 3–6 page section:

```
{
  "sectionId": "02-3-b-trees-vs-lsm",
  "sectionTitle": "2.3 B-Trees vs. LSM-Trees",
  "pageStart": 42,
  "pageEnd": 47,
  "pageCount": 6,
  "estimatedMinutesSaved": 18,
  "mustRemember": [
    {
      "title": "Fixed-Size Block Reads vs. Append-Only Writes",
      "explanation": "B-Trees split storage into fixed 4KB–16KB pages and overwrite them in place. LSM-Trees append all
      "ruleOfThumb": "Use LSM-Trees for write-heavy workloads; use B-Trees when you need predictable, fast point reads."
      "page": 43
    }
  ],
  "usefulLater": [
    {
      "topic": "Write Amplification Calculation",
      "note": "One logical write in an LSM-Tree can trigger multiple disk writes during background compaction.",
      "page": 45
    }
  ],
  "schemas": [
    {
      "id": "schema-wal-flow",
      "title": "How a Write Survives a Power Outage",
      "page": 44,
      "extractedImageFile": "assets/page-044-fig-1.png",
```

```
      "animatedFlow": {
        "nodes": [
          { "id": "client", "label": "Client Write Request" },
          { "id": "wal", "label": "Write-Ahead Log (Disk)" },
          { "id": "memtable", "label": "MemTable (RAM)" },
          { "id": "ack", "label": "Success Response" }
        ],
        "steps": [
          { "step": 1, "from": "client", "to": "wal", "caption": "Append raw write to disk log first" },
          { "step": 2, "from": "wal", "to": "memtable", "caption": "Insert key-value into sorted RAM tree" },
          { "step": 3, "from": "memtable", "to": "ack", "caption": "Confirm write to client only after both succeed" }
        ]
      }
    }
  ],
  "codeExamples": [
    {
      "title": "Crash-Safe Key Insertion",
      "language": "rust",
      "page": 46,
      "originalCode": "pub fn put(&mut self, key: &[u8], val: &[u8]) -> Result<()> {\n    self.wal.append(key, val)?;\n
      "lineExplanations": [
        { "lines": "2-3", "why": "Writes to disk and forces a physical flush BEFORE touching memory. If power dies on li
        { "lines": "4", "why": "Updates the fast in-memory structure only after disk safety is guaranteed." }
      ]
    }
  ],
  "exercises": [
    {
      "type": "spot_the_bug",
      "page": 46,
      "scenario": "A developer tried to speed up inserts by rearranging the code below. What breaks in production?",
      "buggyCode": "pub fn put(&mut self, key: &[u8], val: &[u8]) -> Result<()> {\n    self.memtable.insert(key.to_vec()
      "hint": "Look at both the order of operations and what is missing after append().",
      "solution": "Two critical bugs: 1) Updating memory before the disk log means a crash loses data that concurrent re
    }
  ]
}
```

B. System Prompt to Prevent Shallow Summaries

Put this exact prompt inside `src/ai/prompts.ts`:

```
You are a ruthless Technical Book Extractor. You are processing a 3 to 6 page section of a technical book.
Your job is NOT to write a high-level summary. Summaries throw away valuable engineering details.
Your job is COMPLETE TECHNICAL EXTRACTION:

1. DELETE 100% of conversational filler, author anecdotes, historical trivia, and repetitive introductions.
2. EXTRACT 100% of technical rules, numbers, thresholds, architecture mechanics, edge cases, and warnings.
3. PAGE ACCURACY: Every single point, diagram, code block, and exercise MUST include the exact page number from the ---
4. CODE EXAMPLES: Extract every meaningful code example in this section. Never skip code. Explain the "why" behind criti
5. SCHEMAS & FLOWS: Whenever the text references a figure (or a [EXTRACTED_FIGURE] marker appears) or describes a multi-
6. EXERCISES: Create a practical "Spot the Bug" or "What Happens If..." challenge using the exact mechanics taught on th
```

5. User Interface Requirements & Extra Details

1. **Page Coverage Badge & 1-Click Verification:** Every board header displays a pill button: `📄 Covers Pages 42–47 (6 pages)` alongside individual `(p. 44)` tags on every bullet and code block. Clicking any page tag slides out a right-hand drawer showing the original PDF page rendered locally via `pdfjs-dist` so you can cross-check the book in one second.
2. **Hybrid Diagram Viewer:** In the **Schemas** section of the board, each diagram card has two tabs:
   - **Original Book Figure:** Shows the crisp cropped image (`assets/page-044-fig-1.png`).
   - **Interactive Flow Player:** Shows the React Flow boxes with `[◀ Prev]`, `[▶ Play]`, and `[Next ▶]` buttons that light up Step 1, Step 2, Step 3 in sequence with glowing animated connectors.
   - If a diagram wasn't auto-animated from the surrounding text, clicking **"✨ Animate This Schema"** sends just that single cropped `.png` to the AI vision endpoint and adds the interactive flow to the `.board.json` file.
3. **Settings Modal (`⚙ Settings`):**
   - **Theme Switcher:** Dark Mode (default, high-contrast slate), Light Mode (warm paper), or System.

- **API Key Vault:** Separate password fields for Anthropic, Google Gemini, OpenAI, and OpenRouter keys, saved locally via Tauri's encrypted store so keys never touch plain text files.

6. Recommended 4-Step Build Order

1. **Milestone 1 — Shell & Local Slicer (Day 1):** Scaffold the Tauri 2.12 + React + Tailwind project. Build the Drag & Drop Launchpad and wire up `pdfjs-dist` to read a PDF's Table of Contents, stamp `[PAGE X]` markers, and display the Chapter Filter checklist.
2. **Milestone 2 — AI Extraction & File Writer (Day 2):** Connect Vercel AI SDK 7 (`generateObject` / structured outputs) with `boardSchema.ts` and the System Prompt. Have it process one 5-page section and save the `.board.json` and `.md` files into your chosen output path.
3. **Milestone 3 — The 3-Tier Board Viewer (Day 3):** Build `BoardViewer.tsx` with Shiki for side-by-side code blocks, React Flow + Dagre for the step-by-step animated schemas, and the collapsible "Spot the Bug" solution card.
4. **Milestone 4 — Local Figure Cropping & Page Verification Drawer (Day 4):** Add the local image extractor to save book diagrams into `assets/` and add the slide-out original PDF page viewer when clicking any `(p. X)` badge.
